МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №6

з дисципліни **“Функціональне програмування”**

на тему **“ЗАМИКАННЯ ТА ДЕКОРАТОРИ ЯК HOF”**

Варіант 13. Декоратор контролю доступу за токеном

Виконав:
Студент групи ФЕП-23
Маньківський Максим Володимирович

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: навчитися створювати замикання, що утримують стан, і параметризовані декоратори як фабрики декораторів; реалізувати декоратори таймінгу, повторних спроб і кешування з інвалідацією, а за варіантом 13 — декоратор контролю доступу за токеном з інжектованим сервісом перевірки; забезпечити детерміновані тести без реальних затримок.

Теоретичні відомості:

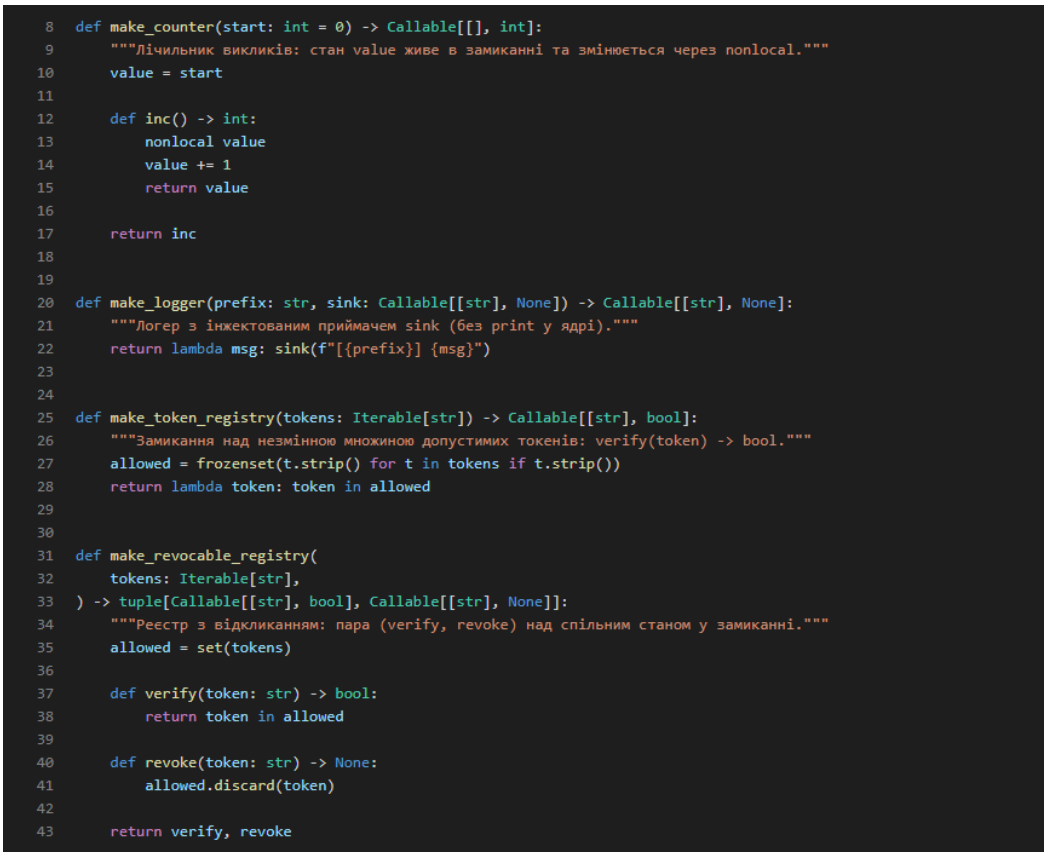
Замикання — це функція, яка «пам'ятає» змінні з навколишньої області видимості навіть після її завершення; через `nonlocal` внутрішня функція може змінювати захоплений стан (лічильник), а без нього — просто читати його (префікс логера, множина допустимих токенів). Функції вищого порядку приймають або повертають функції, і декоратор — класичний випадок: `decorator(func)` повертає обгорнуту версію `func`. Параметризований декоратор — це фабрика `factory(*opts)`, що повертає звичайний декоратор; ланцюжок `factory → decorator → wrapper` працює саме завдяки замиканням.

Обов'язковий `functools.wraps` зберігає `__name__`, `__doc__` та `__wrapped__` обгорнутої функції. Для типізації обгорток використовують `ParamSpec` і `TypeVar`, а якщо обгортка додає аргумент — `Concatenate`. Зовнішні залежності (годинник, `sleep`, логер, джерело часу, сервіс перевірки) передають у декоратор як `Callable`: тоді ядро лишається чистим, а тести підміняють їх фейками і не чекають реального часу. Декоратори застосовуються знизу вгору, тому порядок визначає поведінку: кеш усередині авторизації не дозволяє обійти перевірку токена кеш-хітом.

Хід роботи:

1. У `closures.py` реалізував замикання: `make_counter` тримає змінний стан через `nonlocal`, `make_logger` захоплює префікс і інжектований приймач `sink` (без `print` у ядрі), `make_token_registry` — замикання над незмінною множиною

токенів для варіанта 13, а `make_revocable_registry` повертає пару функцій `verify` / `revoke` над спільним станом:



```python
def make_counter(start: int = 0) -> Callable[[], int]:
    """Лічильник викликів: стан value живе в замиканні та змінюється через nonlocal."""
    value = start

    def inc() -> int:
        nonlocal value
        value += 1
        return value

    return inc


def make_logger(prefix: str, sink: Callable[[str], None]) -> Callable[[str], None]:
    """Логер з інжектованим приймачем sink (без print у ядрі)."""
    return lambda msg: sink(f"[{prefix}] {msg}")


def make_token_registry(tokens: Iterable[str]) -> Callable[[str], bool]:
    """Замикання над незмінною множиною допустимих токенів: verify(token) -> bool."""
    allowed = frozenset(t.strip() for t in tokens if t.strip())
    return lambda token: token in allowed


def make_revocable_registry(
    tokens: Iterable[str],
) -> tuple[Callable[[str], bool], Callable[[str], None]]:
    """Реєстр з відкликанням: пара (verify, revoke) над спільним станом у замиканні."""
    allowed = set(tokens)

    def verify(token: str) -> bool:
        return token in allowed

    def revoke(token: str) -> None:
        allowed.discard(token)

    return verify, revoke
```

2. У `decorators.py` описав типи: `ParamSpec` / `TypeVar`, власний виняток `AuthError`, протоколи `Decorator`, `CacheDecorator`, `AuthDecorator` з узагальненим

`__call__` (щоб `mypy --strict` виводив сигнатуру обгорнутої функції) і протокол `CachedFn` з методами керування кешем:

```python
P = ParamSpec("P")
R = TypeVar("R")
R_co = TypeVar("R_co", covariant=True)

Key = tuple[tuple[Any, ...], tuple[tuple[str, Any], ...]]


class AuthError(PermissionError):
    """Токен відсутній, порожній або не пройшов перевірку."""


class Decorator(Protocol):
    """Декоратор, що зберігає сигнатуру: Callable[P, R] -> Callable[P, R]."""

    def __call__(self, func: Callable[P, R], /) -> Callable[P, R]: ...


class CachedFn(Protocol[P, R_co]):
    """Обгортка з кешем: звичайний виклик плюс методи керування кешем."""

    __name__: str
    __doc__: str | None

    def __call__(self, *args: P.args, **kwargs: P.kwargs) -> R_co: ...

    def cache_clear(self) -> None: ...

    def cache_invalidate(
        self, predicate: Callable[[tuple[Any, ...], dict[str, Any]], bool]
    ) -> int: ...

    def cache_size(self) -> int: ...


class CacheDecorator(Protocol):
    """Декоратор, що повертає обгортку з методами кешу."""

    def __call__(self, func: Callable[P, R], /) -> CachedFn[P, R]: ...


class AuthDecorator(Protocol):
    """Декоратор, що додає токен першим позиційним аргументом."""

    def __call__(self, func: Callable[P, R], /) -> Callable[Concatenate[str, P], R]: ...

```

3. Реалізував параметризований декоратор `timed`: мітка, одиниці виміру, інжектовані `logger` і `clock`; вимір у блоці `finally`, тому час логується навіть при

винятку; `functools.wraps` зберігає метадані:

```python
def timed(
    label: str | None = None,
    *,
    unit: Literal["s", "ms", "us"] = "ms",
    logger: Callable[[str], None],
    clock: Callable[[], float] = time.perf_counter,
) -> Decorator:
    """Логує час виконання (навіть при винятку); годинник і логер інжектовані."""
    factor = {"s": 1.0, "ms": 1e3, "us": 1e6}[unit]

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        name = label or func.__name__

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            t0 = clock()
            try:
                return func(*args, **kwargs)
            finally:
                logger(f"[timed] {name} took {(clock() - t0) * factor:.3f} {unit}")

        return wrapper

    return decorator

```

4. Реалізував `retries` з експоненційним бекофом і джиттером: кортеж винятків, на які реагуємо, інжектовані `sleep`, `logger` і генератор випадкових чисел `rand`; після останньої невдалої спроби виняток піднімається повторно:

```python
def retries(
    attempts: int = 3,
    exceptions: tuple[type[BaseException], ...] = (Exception,),
    delay: float = 0.0,
    backoff: float = 1.0,
    jitter: float = 0.0,
    *,
    sleep: Callable[[float], None] = time.sleep,
    logger: Callable[[str], None] = lambda _: None,
    rand: Callable[[], float] = random.random,
) -> Decorator:
    """Повторює виклик при зазначених винятках з експоненційним бекофом і джиттером."""
    if attempts < 1:
        raise ValueError("attempts must be >= 1")

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            for i in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    if i == attempts:
                        logger(f"[retries] fail after {i} attempts: {e!r}")
                        raise
                    wait = delay * backoff ** (i - 1) + (
                        rand() * jitter if jitter else 0.0
                    )
                    logger(f"[retries] attempt {i} failed: {e!r}; sleeping {wait:.3f}s")
                    sleep(wait)
            raise AssertionError("unreachable")  # pragma: no cover

        return wrapper

    return decorator

```

5. Реалізував `cached` з TTL: ключ з позиційних і відсортованих іменованих аргументів, протухання за інжектованим `time_fn`, блокування для безпеки у

потоках, а на обгортці — `cache_clear`, `cache_invalidate(predicate)` і `cache_size`; результат приводиться до протоколу `CachedFn` через `cast`:

```python
def make_key(args: tuple[Any, ...], kwargs: dict[str, Any]) -> Key:
    """Ключ кешу для хешованих аргументів: (args, відсортовані kwargs)."""
    return (args, tuple(sorted(kwargs.items())))


def cached(
    *,
    ttl: float | None = None,
    time_fn: Callable[[], float] = time.time,
) -> CacheDecorator:
    """Кеш з TTL; на обгортці є cache_clear, cache_invalidate, cache_size."""

    def decorator(func: Callable[P, R]) -> CachedFn[P, R]:
        cache: dict[Key, tuple[float | None, R]] = {}
        lock = threading.Lock()  # ponytail: один лок на функцію, достатньо для ЛР

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            key = make_key(args, kwargs)
            now = time_fn()
            with lock:
                if key in cache:
                    expiry, value = cache[key]
                    if expiry is None or now < expiry:
                        return value
                    del cache[key]
            value = func(*args, **kwargs)
            with lock:
                cache[key] = (None if ttl is None else now + ttl, value)
            return value

        def cache_clear() -> None:
            with lock:
                cache.clear()

        def cache_invalidate(
            predicate: Callable[[tuple[Any, ...], dict[str, Any]], bool],
        ) -> int:
            with lock:
                doomed = [k for k in cache if predicate(k[0], dict(k[1]))]
                for k in doomed:
                    del cache[k]
            return len(doomed)

        def cache_size() -> int:
            with lock:
                return len(cache)

        wrapper.cache_clear = cache_clear  # type: ignore[attr-defined]
        wrapper.cache_invalidate = cache_invalidate  # type: ignore[attr-defined]
        wrapper.cache_size = cache_size  # type: ignore[attr-defined]
        return cast(CachedFn[P, R], wrapper)

    return decorator

```

6. За варіантом 13 реалізував `authorized(verify)`: обгортка вимагає токен першим позиційним аргументом (тип `Callable[Concatenate[str, P], R]`),

відхиляє порожній або невалідний токен через `AuthError` і не передає токен у захищену функцію; сервіс перевірки інжектується як `Callable[[str], bool]`:

```python
def authorized(
    verify: Callable[[str], bool],
    *,
    logger: Callable[[str], None] = lambda _: None,
) -> AuthDecorator:
    """Варіант 13: обгортка вимагає токен ПЕРШИМ аргументом і пропускає виклик лише
    з валідним токеном; сама захищена функція токена не бачить.

    verify — інжектований сервіс перевірки (замикання над реєстром або віддалений
    сервіс, обгорнутий у @retries). Типізовано через Concatenate[str, P].
    """

    def decorator(func: Callable[P, R]) -> Callable[Concatenate[str, P], R]:
        @functools.wraps(func)
        def wrapper(token: str, /, *args: P.args, **kwargs: P.kwargs) -> R:
            if not token:
                logger(f"[auth] {func.__name__}: token missing")
                raise AuthError("token required")
            if not verify(token):
                logger(f"[auth] {func.__name__}: token rejected")
                raise AuthError("invalid token")
            logger(f"[auth] {func.__name__}: ok")
            return func(*args, **kwargs)

        return wrapper

    return decorator
```

7. В `app.py` зібрав демонстрацію: нестабільний `remote_verify` під `@retries` з фейковим `sleep`, дорога операція під `@cached` і композиція

`timed(authorized(cached(…)))`; годинник детермінований через замикання-лічильник:

```python
log = make_logger("app", print)
ticks = make_counter()
fake_clock = lambda: ticks() * 0.001  # noqa: E731  — детермінований «годинник»
fake_sleep = lambda s: log(f"(sleep {s:.3f}s skipped)")  # noqa: E731

local_verify = make_token_registry(["alpha-1", "beta-2"])
flaky_calls = make_counter()


@retries(attempts=3, delay=0.1, backoff=2.0, sleep=fake_sleep, logger=log)
def remote_verify(token: str) -> bool:
    """Нестабільний сервіс перевірки: перші два виклики падають, далі відповідає."""
    if flaky_calls() < 3:
        raise ConnectionError("verify service unavailable")
    return local_verify(token)


@cached(ttl=60.0, time_fn=fake_clock)
def compute_balance(account_id: int) -> float:
    """Дорога операція з кешем (внутрішній шар, без контролю доступу)."""
    log(f"computing balance for {account_id}")
    return 100.0 * account_id

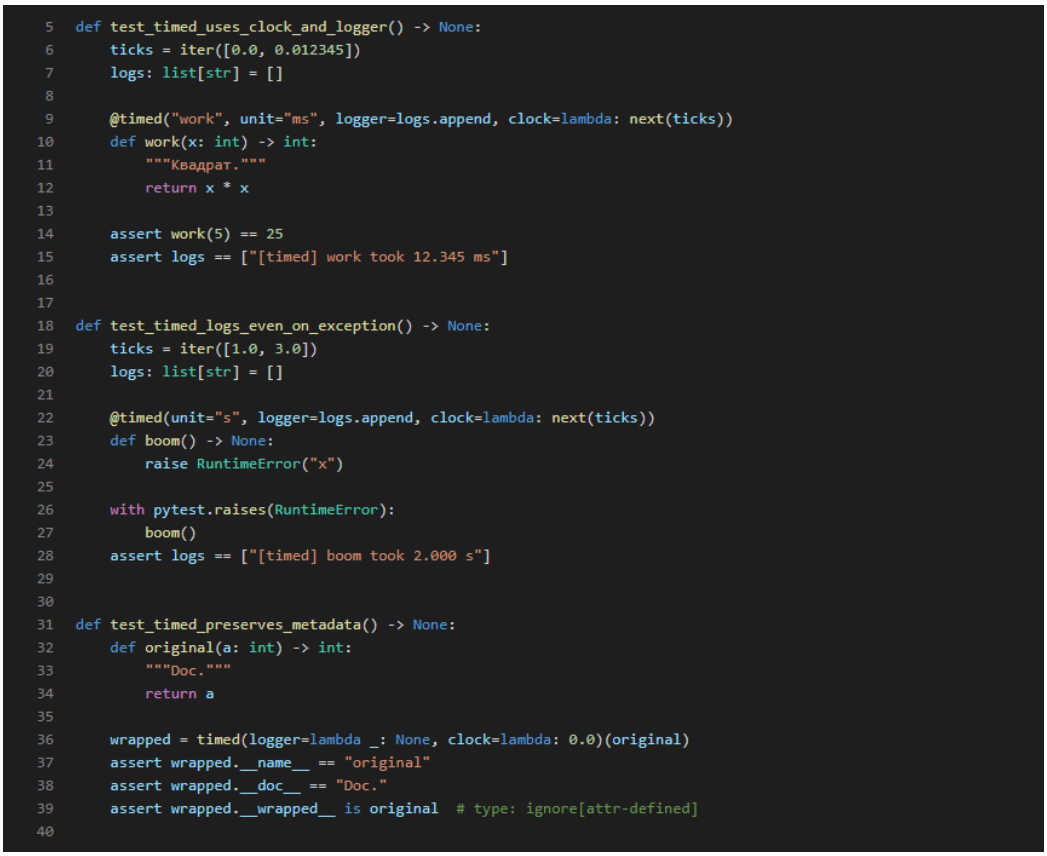

# порядок знизу вгору: кеш -> авторизація -> таймінг; кеш-хіт усе одно вимагає токен
account_balance = timed("balance", logger=log, clock=fake_clock)(
    authorized(remote_verify, logger=log)(compute_balance)
)


def main() -> None:
    """Показує успішний виклик, кеш-хіт і відмову за поганим токеном."""
    print("balance:", account_balance("alpha-1", 7))
    print("balance again (cache hit):", account_balance("beta-2", 7))
    print("cache size:", compute_balance.cache_size())
    for bad in ("", "nope"):
        try:
            account_balance(bad, 7)
        except AuthError as e:
            print("denied:", e)

```

8. Написав тести `timed`: фейковий годинник і список-логер замість реального

часу, лог при винятку, збереження `__name__`, `__doc__` та `__wrapped__`:



```python
def test_timed_uses_clock_and_logger() -> None:
    ticks = iter([0.0, 0.012345])
    logs: list[str] = []

    @timed("work", unit="ms", logger=logs.append, clock=lambda: next(ticks))
    def work(x: int) -> int:
        """Квадрат."""
        return x * x

    assert work(5) == 25
    assert logs == ["[timed] work took 12.345 ms"]


def test_timed_logs_even_on_exception() -> None:
    ticks = iter([1.0, 3.0])
    logs: list[str] = []

    @timed(unit="s", logger=logs.append, clock=lambda: next(ticks))
    def boom() -> None:
        raise RuntimeError("x")

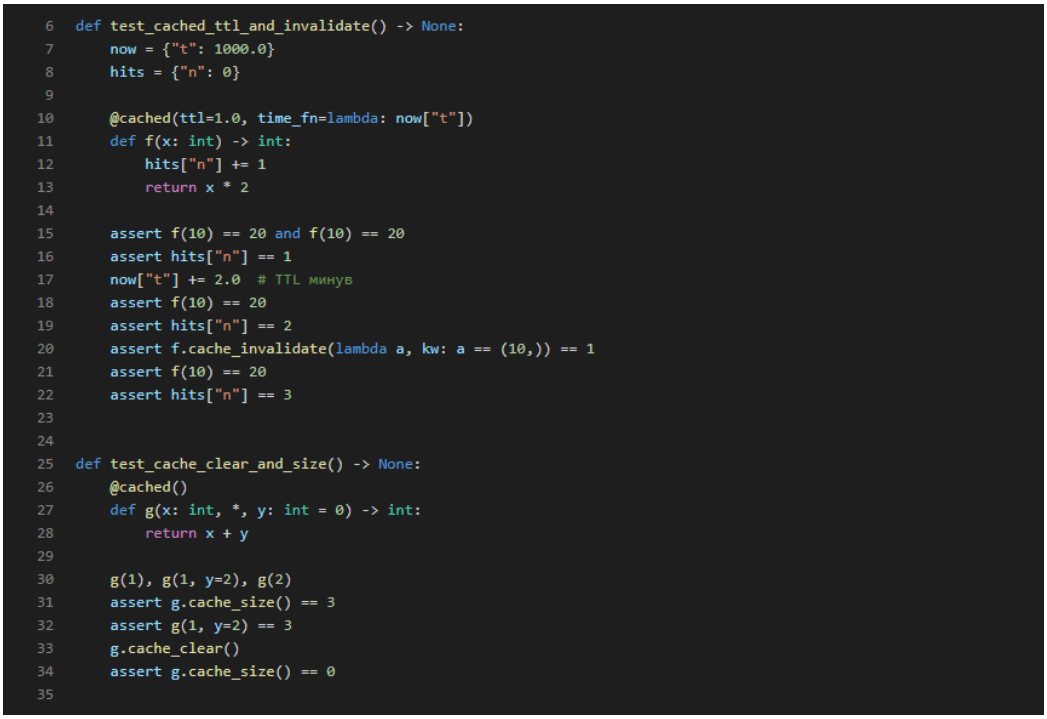
    with pytest.raises(RuntimeError):
        boom()
    assert logs == ["[timed] boom took 2.000 s"]


def test_timed_preserves_metadata() -> None:
    def original(a: int) -> int:
        """Doc."""
        return a

    wrapped = timed(logger=lambda _: None, clock=lambda: 0.0)(original)
    assert wrapped.__name__ == "original"
    assert wrapped.__doc__ == "Doc."
    assert wrapped.__wrapped__ is original  # type: ignore[attr-defined]

```

9. Тести `cached`: TTL через підмінений `time_fn`, точкова інвалідація за предикатом, повне очищення та розмір кешу; тести `retries` перевіряють успіх на третій спробі без реального сну, бекоф `[0.5, 1.0]`, реакцію лише на вказані винятки та детермінований джиттер:



```python
def test_cached_ttl_and_invalidate() -> None:
    now = {"t": 1000.0}
    hits = {"n": 0}

    @cached(ttl=1.0, time_fn=lambda: now["t"])
    def f(x: int) -> int:
        hits["n"] += 1
        return x * 2

    assert f(10) == 20 and f(10) == 20
    assert hits["n"] == 1
    now["t"] += 2.0  # TTL минув
    assert f(10) == 20
    assert hits["n"] == 2
    assert f.cache_invalidate(lambda a, kw: a == (10,)) == 1
    assert f(10) == 20
    assert hits["n"] == 3


def test_cache_clear_and_size() -> None:
    @cached()
    def g(x: int, *, y: int = 0) -> int:
        return x + y

    g(1), g(1, y=2), g(2)
    assert g.cache_size() == 3
    assert g(1, y=2) == 3
    g.cache_clear()
    assert g.cache_size() == 0

```

10. Тести варіанта 13: валідний, порожній і чужий токен, сервіс перевірки з ретраями (два падіння `ConnectionError`, потім відповідь) і порядок декораторів — кеш-хіт при валідному токені, відмова без звернення в кеш при невалідному, `timed` на кожен виклик:

```python
def test_verify_service_with_retries_is_injected() -> None:
    attempts = make_counter()
    slept: list[float] = []

    @retries(attempts=3, delay=0.1, sleep=slept.append)
    def remote_verify(token: str) -> bool:
        if attempts() < 3:
            raise ConnectionError("down")
        return token == "ok"

    @authorized(remote_verify)
    def secret() -> str:
        return "data"

    assert secret("ok") == "data"
    assert slept == [0.1, 0.1]
    with pytest.raises(AuthError):
        secret("bad")


def test_decorator_order_cache_inside_auth() -> None:
    hits = make_counter()
    logs: list[str] = []
    tick = make_counter()

    @cached()
    def compute(x: int) -> int:
        hits()
        return x * 10

    guarded = timed("op", logger=logs.append, clock=lambda: float(tick()))(
        authorized(make_token_registry(["t"]))(compute)
    )
    assert guarded("t", 2) == 20
    assert guarded("t", 2) == 20
    assert compute.cache_size() == 1 and hits() == 2  # кеш-хіт: обчислено один раз
    with pytest.raises(AuthError):
        guarded("no", 2)  # кеш не обходить авторизацію
    assert len(logs) == 3  # timed спрацював на кожен виклик, включно з відмовою
```

11. Запустив демонстрацію: сервіс перевірки «падає» двічі й відновлюється без реальних затримок, другий виклик бере баланс із кешу, а порожній і чужий токени відхиляються з логуванням часу:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab06> python app.py
[app] [retries] attempt 1 failed: ConnectionError('verify service unavailable'); sleeping 0.100s
[app] (sleep 0.100s skipped)
[app] [retries] attempt 2 failed: ConnectionError('verify service unavailable'); sleeping 0.200s
[app] (sleep 0.200s skipped)
[app] [auth] compute_balance: ok
[app] computing balance for 7
[app] [timed] balance took 2.000 ms
balance: 700.0
[app] [auth] compute_balance: ok
[app] [timed] balance took 2.000 ms
balance again (cache hit): 700.0
cache size: 1
[app] [auth] compute_balance: token missing
[app] [timed] balance took 1.000 ms
denied: token required
[app] [auth] compute_balance: token rejected
[app] [timed] balance took 1.000 ms
denied: invalid token
```

12. Перевірив проєкт автоматичними інструментами: усі 27 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають зауважень до стилю:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab06> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs\FunctionalProgramming
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 27 items

tests\test_authorized.py::test_authorized_allows_valid_token_and_hides_it PASSED [  3%]
tests\test_authorized.py::test_authorized_rejects_bad_tokens[] PASSED    [  7%]
tests\test_authorized.py::test_authorized_rejects_bad_tokens[wrong] PASSED [ 11%]
tests\test_authorized.py::test_authorized_rejects_bad_tokens[GOOD] PASSED [ 14%]
tests\test_authorized.py::test_authorized_error_messages PASSED          [ 18%]
tests\test_authorized.py::test_verify_service_with_retries_is_injected PASSED [ 22%]
tests\test_authorized.py::test_decorator_order_cache_inside_auth PASSED  [ 25%]
tests\test_cached.py::test_cached_ttl_and_invalidate PASSED              [ 29%]
tests\test_cached.py::test_cache_clear_and_size PASSED                   [ 33%]
tests\test_cached.py::test_cached_preserves_metadata PASSED              [ 37%]
tests\test_cached.py::test_property_each_distinct_arg_computed_once PASSED [ 40%]
tests\test_closures.py::test_counter_keeps_state PASSED                  [ 44%]
tests\test_closures.py::test_counters_are_independent PASSED             [ 48%]
tests\test_closures.py::test_logger_uses_injected_sink PASSED            [ 51%]
tests\test_closures.py::test_token_registry PASSED                       [ 55%]
tests\test_closures.py::test_revocable_registry_shares_state PASSED      [ 59%]
tests\test_closures.py::test_property_counter_counts_calls PASSED        [ 62%]
tests\test_closures.py::test_property_registry_is_exact_membership PASSED [ 66%]
tests\test_retries.py::test_retries_succeeds_on_third_attempt_without_sleep PASSED [ 70%]
tests\test_retries.py::test_retries_reraises_after_last_attempt PASSED   [ 74%]
tests\test_retries.py::test_retries_only_for_listed_exceptions PASSED    [ 77%]
tests\test_retries.py::test_retries_jitter_is_deterministic_with_injected_rand PASSED [ 81%]
tests\test_retries.py::test_retries_invalid_attempts PASSED              [ 85%]
tests\test_timed.py::test_timed_uses_clock_and_logger PASSED             [ 88%]
tests\test_timed.py::test_timed_logs_even_on_exception PASSED            [ 92%]
tests\test_timed.py::test_timed_preserves_metadata PASSED                [ 96%]
tests\test_timed.py::test_timed_units PASSED                             [100%]

============================= 27 passed in 1.37s ==============================

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab06> python -m mypy .
Success: no issues found in 9 source files

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab06> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab06> python -m black --check .
All done! ✨ 🍰 ✨
9 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я реалізував замикання зі станом (`nonlocal`) і з інжектованими залежностями, параметризовані декоратори `timed`, `retries` і `cached` з TTL та інвалідацією, а за варіантом 13 — декоратор `authorized`, що перевіряє токен через інжектований сервіс і типізований через `Concatenate`. Усі зовнішні ефекти (час, сон, логування, перевірка токена) передаються як `Callable`, тому 27 тестів виконуються детерміновано без реальних затримок і підтверджують збереження метаданих через `functools.wraps`, роботу винятків і граничних параметрів та вплив порядку декораторів; mypy, ruff і black пройшли без зауважень.